

БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
Факультет прикладной информатики и компьютерных технологий
Кафедра информационных технологий

КУРСОВАЯ РАБОТА

по дисциплине «Проектирование информационных систем»

на тему:

**«Разработка системы автоматизированного сбора и первичной
обработки данных из внешнего API с использованием Apache Airflow и
объектного хранилища»**

Выполнил:

Руководитель:

Группа:

Дата защиты:

студент

Л. Судиловский

«___» _____ 2026

г.

Минск, 2026

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

Тема работы: «Разработка системы автоматизированного сбора и первичной обработки данных из внешнего API с использованием Apache Airflow и объектного хранилища».

Исходные данные: репозиторий `course_project`, учебный проект `pet_project_earthquake`, документация Apache Airflow, DuckDB, MinIO и OpenSky Network API, а также архитектурная схема ETL-конвейера.

Цель работы: разработать и описать систему, которая по расписанию получает данные из внешнего API, выполняет первичную нормализацию результата и сохраняет данные в S3-совместимое объектное хранилище в формате Parquet.

Основные задачи:

- проанализировать предметную область автоматизированного сбора данных и особенности внешнего API;
- обосновать выбор Apache Airflow, DuckDB, MinIO, Docker Compose, PostgreSQL и Redis;
- спроектировать архитектуру ETL-конвейера и описать протоколы взаимодействия компонентов;
- реализовать DAG Airflow для загрузки данных OpenSky Network API в объектное хранилище;
- описать вопросы конфигурации, безопасности, идиоматичности, качества и эксплуатации решения.

Ожидаемый результат: работоспособный локальный прототип data engineering системы, в котором Apache Airflow управляет запуском задач, DuckDB читает JSON из внешнего API и записывает Parquet-файлы в S3-совместимое хранилище MinIO.

ОГЛАВЛЕНИЕ

Оглавление обновляется автоматически в Microsoft Word/LibreOffice

ВВЕДЕНИЕ

Современные информационные системы всё чаще строятся вокруг регулярного обмена данными между внешними сервисами, внутренними аналитическими платформами и хранилищами. Для организаций, которые принимают решения на основе данных, важно не только получить сведения из внешнего API, но и сделать этот процесс повторяемым, контролируемым и воспроизводимым. Ручная загрузка файлов или разовые скрипты быстро становятся источником ошибок: сложно гарантировать периодичность запуска, трудно отследить сбои, а результаты разных запусков могут храниться в несогласованном виде.

Тема данной курсовой работы посвящена разработке системы автоматизированного сбора и первичной обработки данных из внешнего API с использованием Apache Airflow и объектного хранилища. Практическая часть реализована на основе учебного data engineering проекта, однако область решения намеренно сужена до одного завершённого этапа — загрузки сырых данных из OpenSky Network API в S3-совместимое хранилище MinIO. Такой подход позволяет сосредоточиться на наиболее важной для дальнейшей аналитики части конвейера: надёжном извлечении данных и сохранении исходного слоя в формате, пригодном для последующей обработки.

Актуальность работы определяется ростом роли автоматизированных ETL/ELT-процессов. Данные из внешних API могут изменяться каждую минуту, иметь ограничения по доступности, задержки ответа и нестабильную структуру. Поэтому система сбора должна включать планировщик, механизм повторных попыток, изолированную инфраструктуру, единый формат хранения и безопасное управление ключами доступа. В проекте эти задачи решаются средствами Apache Airflow, Docker Compose, DuckDB и MinIO.

Объектом исследования является процесс автоматизированного сбора данных из внешнего REST API. Предметом исследования является

программная и инфраструктурная реализация конвейера, который получает данные OpenSky Network API, выполняет первичное преобразование JSON-ответа и сохраняет результат в объектном хранилище в формате Parquet.

Цель курсовой работы — разработать и описать систему автоматизированного сбора и первичной обработки данных из внешнего API с использованием Apache Airflow и S3-совместимого объектного хранилища. Для достижения цели необходимо решить следующие задачи: изучить предметную область и требования к сбору данных; выбрать технологический стек; спроектировать архитектуру взаимодействия компонентов; реализовать DAG Airflow; описать хранение, безопасность, качество данных и эксплуатацию решения; оценить результат и направления развития проекта.

Практическая значимость работы заключается в том, что полученный прототип может быть использован как основа для дальнейшего развития учебного или прикладного data engineering проекта. Слой raw/bronze, сформированный в объектном хранилище, может стать источником для последующих этапов: загрузки в реляционную базу данных, построения витрин, проверки качества и визуализации.

1 Теоретические основы автоматизированного сбора данных

1.1 Понятие автоматизированного сбора данных

Автоматизированный сбор данных представляет собой процесс регулярного получения информации из внешних или внутренних источников без участия пользователя в каждом отдельном запуске. В отличие от ручной выгрузки, автоматизированный подход предполагает наличие расписания, единых правил обработки, журналирования и механизма реакции на ошибки. В data engineering такой процесс обычно рассматривается как часть ETL- или ELT-конвейера, где Extract отвечает за извлечение, Transform — за преобразование, а Load — за загрузку в целевое хранилище.

В рассматриваемом проекте реализована часть Extract и первичного Load. Система обращается к OpenSky Network API, получает актуальные

сведения о воздушных судах, разворачивает вложенную структуру ответа и сохраняет результат в объектное хранилище. Преобразования на данном этапе минимальны: данные сохраняются максимально близко к исходному виду. Это соответствует практике построения lakehouse-архитектур, где raw- или bronze-слой содержит исходные сведения и служит неизменяемой точкой восстановления для будущих обработок.

Ключевые требования к такой системе включают регулярность, воспроизводимость, наблюдаемость и устойчивость к ошибкам. Регулярность достигается расписанием DAG. Воспроизводимость обеспечивается контейнерной инфраструктурой и фиксированными версиями зависимостей. Наблюдаемость реализуется средствами веб-интерфейса Airflow и журналами выполнения задач. Устойчивость повышается за счёт retry-механизма, ограничения параллельности и разнесения компонентов по сервисам Docker Compose.

1.2 Внешний API как источник данных

Внешний API является интерфейсом, через который сторонний сервис предоставляет данные другим приложениям. OpenSky Network API публикует информацию о воздушном пространстве и состоянии воздушных судов. Согласно документации OpenSky, данные представляются в виде state vectors — векторов состояния воздушных судов, содержащих идентификатор, временные метки, координаты, скорость, курс и другие параметры. Ответ метода /api/states/all возвращается в формате JSON, что удобно для первичного получения, но требует нормализации перед долгосрочным хранением.

Особенность такого источника состоит в том, что он описывает текущее состояние, а не историческую таблицу с заранее известным набором строк. Это означает, что при каждом запуске конвейера получает снимок состояния на момент обращения. Для учебного проекта такой источник хорошо демонстрирует типичную задачу data engineering: внешний сервис

живёт независимо от локальной системы, а значит, конвейер должен учитывать сетевое взаимодействие, возможные задержки, ограничения API и необходимость сохранять результат запуска с привязкой к дате интервала.

Формат JSON удобен для обмена между сервисами, но для аналитического хранения часто менее эффективен, чем колоночные форматы. Поэтому в проекте после чтения JSON используется запись в Parquet. Parquet обеспечивает компактное хранение и эффективное чтение колонок при последующей аналитической обработке. Сохранение результата в S3-совместимое хранилище делает данные доступными для разных инструментов: DuckDB, Spark, Trino, Python-скриптов и BI-систем.

1.3 Apache Airflow как оркестратор

Apache Airflow — платформа для программного описания, планирования и мониторинга рабочих процессов. Основным объектом Airflow — DAG, то есть направленный ациклический граф задач. DAG задаёт расписание, набор задач и зависимости между ними. Каждая задача выполняет отдельную единицу работы, а Airflow отвечает за запуск по расписанию, хранение статусов, повторные попытки и предоставление интерфейса мониторинга.

В данной работе Airflow выбран потому, что он хорошо подходит для регулярных data engineering процессов. DAG `raw_from_api_to_s3` содержит три логических шага: начальный маркер `start`, задачу `get_and_transfer_api_data_to_s3` и конечный маркер `end`. Центральная задача реализована через `PythonOperator`, что позволяет использовать Python-функцию для запуска SQL-команд DuckDB. Такой подход сочетает гибкость Python и эффективность DuckDB при работе с внешними файлами и объектным хранилищем.

Для локального запуска используется `CeleryExecutor`. Он требует брокера сообщений Redis и базы метаданных PostgreSQL. PostgreSQL хранит информацию о DAG, задачах, запусках и пользователях, а Redis используется

как очередь задач между планировщиком и worker-процессами. Несмотря на то что для учебного проекта можно было бы использовать более простой executor, CeleryExecutor демонстрирует архитектуру, близкую к промышленным установкам Airflow, где задачи могут распределяться между несколькими worker-узлами.

1.4 Объектное хранилище и формат Parquet

Объектное хранилище отличается от файловой системы и реляционной базы тем, что данные размещаются в виде объектов внутри бакетов. Каждый объект имеет ключ, содержимое и метаданные. S3-совместимый интерфейс стал фактическим стандартом для data lake решений, поскольку позволяет хранить большие объёмы файлов и подключать к ним разные вычислительные инструменты. В проекте роль такого хранилища выполняет MinIO — локальное S3-совместимое решение, разворачиваемое в контейнере.

В MinIO создаётся bucket bronze, который соответствует первичному слою хранилища. Внутри бакета данные организуются по пути raw/opensky/<дата>/<файл>. Такая структура облегчает навигацию, позволяет отделить источник от слоя обработки и создаёт основу для партиционирования по дате. Даже если в дальнейшем появятся другие источники, их можно будет разместить рядом, сохранив единый принцип каталогизации.

Формат Parquet выбран как колоночный формат хранения. Он удобен для аналитических нагрузок, потому что позволяет читать только нужные столбцы, поддерживает сжатие и хорошо интегрируется с современными инструментами обработки данных. В рассматриваемом DAG запись в Parquet выполняется командой DuckDB COPY TO, направленной в S3-путь. DuckDB через расширение httpfs выступает как лёгкий движок первичной обработки: он читает JSON из API, преобразует вложенный массив states в строки и записывает результат в объектное хранилище.

1.5 ETL, ELT и место raw-слоя в архитектуре

В классическом подходе ETL данные сначала извлекаются из источника, затем преобразуются и после этого загружаются в целевую систему. В ELT данные сначала помещаются в хранилище, а преобразования выполняются уже внутри аналитической платформы. Для современных data lake и lakehouse решений чаще используется логика ELT, поскольку объектное хранилище позволяет недорого сохранять исходные данные и возвращаться к ним при изменении требований.

Рассматриваемая система ближе к ELT-подходу. DAG выполняет извлечение из внешнего API и минимальную техническую нормализацию, необходимую для сохранения результата в Parquet. Бизнес-правила, агрегации и витрины намеренно не реализуются на этом этапе. Это снижает риск потери исходной информации: если в будущем изменится логика расчётов, можно перечитать raw-слой и построить новые производные наборы данных.

Raw-слой выполняет роль доверенного архива поступивших данных. Он должен быть организован так, чтобы можно было понять источник, дату получения и формат файла. Поэтому в проекте путь содержит слой raw, название источника opensky и дату интервала. Такая схема каталогов проста, но уже соответствует распространённой практике построения зон данных: bronze/raw для исходных файлов, silver/ods для очищенных и типизированных данных, gold/dm для витрин и показателей.

Даже если курсовой проект ограничивается первым этапом, выбранная структура не закрывает путь к развитию. Добавление новых DAG может происходить без изменения уже сохранённых объектов. Например, отдельный DAG может читать s3://bronze/raw/opensky, приводить массив state к именованным колонкам и сохранять результат в silver-слой. Другой DAG может строить агрегаты по странам происхождения, высотам, скоростям или географическим областям. Таким образом, корректная организация raw-слоя является фундаментом всей последующей аналитики.

2 Проектирование системы автоматизированной загрузки данных

2.1 Общая архитектура решения

Архитектура разработанного решения представляет собой локальный ETL-конвейер, развернутый с помощью Docker Compose. В центре архитектуры находится Apache Airflow, который управляет расписанием и выполнением DAG. Внешним источником данных является OpenSky Network API, доступный по HTTPS. Целевым хранилищем выступает MinIO, предоставляющий S3 API поверх HTTP. Для внутренних нужд Airflow используются PostgreSQL и Redis.

Пользователь взаимодействует с системой через два веб-интерфейса. Airflow UI доступен на порту 8080 и используется для просмотра DAG, запусков, логов и статусов задач. MinIO Console доступна на порту 9001 и позволяет просматривать бакеты и сохранённые объекты. Непосредственная запись данных выполняется не через браузер, а worker-компонентом Airflow, внутри которого запускается PythonOperator и DuckDB.

Главная особенность архитектуры состоит в разделении ролей. Airflow отвечает за оркестрацию, DuckDB — за чтение и запись данных, MinIO — за хранение, PostgreSQL — за метаданные Airflow, Redis — за распределение задач Celery. Такое разделение делает систему понятной и расширяемой: каждый компонент можно заменить или масштабировать независимо, не меняя концепцию конвейера.

Архитектура ETL-конвейера и протоколы взаимодействия

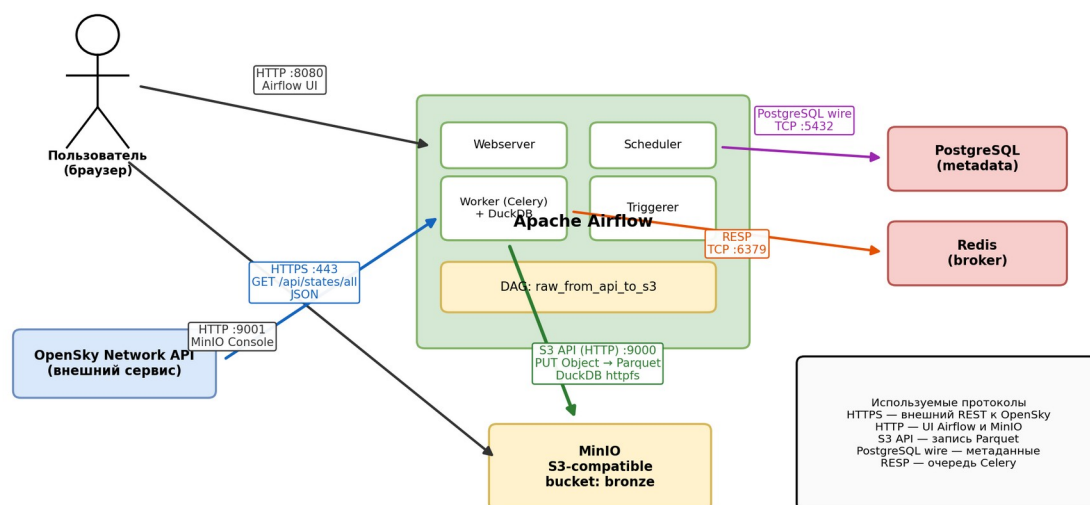


Рисунок 1 – Архитектура ETL-конвейера и протоколы взаимодействия компонентов

2.2 Состав Docker-инфраструктуры

Файл `docker-compose.yaml` основан на официальном шаблоне Apache Airflow 2.10.5 для CeleryExecutor. В проекте определены сервисы `minio`, `postgres`, `redis`, `airflow-webserver`, `airflow-scheduler`, `airflow-worker`, `airflow-triggerer`, `airflow-init`, `airflow-cli` и опциональный `flower`. Также в исходном учебном проекте присутствовали PostgreSQL для DWH и Metabase, однако в курсовой реализации они не используются и закомментированы, поскольку тема ограничена автоматической загрузкой данных в S3.

Сервис `minio` использует образ `minio/minio` и публикует порты 9000 и 9001. Порт 9000 предназначен для S3 API, а 9001 — для консоли управления. Для учебного запуска в `docker-compose` указаны локальные демонстрационные учётные данные `minioadmin/minioadmin`. Доступ DAG к S3 не зашивается в код, а берётся из Variables Airflow: `access_key` и `secret_key`. В курсовой работе реальные значения ключей не раскрываются, так как они относятся к конфигурации окружения.

Сервис `postgres` хранит метаданные Airflow и поднимается на базе PostgreSQL 13. Redis 7.2-bookworm используется как брокер Celery. Airflow

webserver предоставляет пользовательский интерфейс; scheduler анализирует DAG и создаёт запуски; worker выполняет задачи; triggerer предназначен для асинхронных операций и входит в стандартную инфраструктуру Airflow. airflow-init выполняет миграции базы и создаёт начального пользователя веб-интерфейса.

Таблица 1 – Компоненты программной инфраструктуры

Компонент	Назначение	Порт/ протокол	Роль в проекте
Apache Airflow Webserver	Веб-интерфейс управления DAG	HTTP 8080	Просмотр запусков, логов и статусов
Apache Airflow Scheduler	Планирование запусков	PostgreSQL wire	Создаёт DAG run по расписанию
Apache Airflow Worker	Выполнение задач	RESP, HTTPS, S3 API	Запускает PythonOperator и DuckDB
PostgreSQL	База метаданных Airflow	TCP 5432	Хранит статусы, DAG run, пользователей
Redis	Брокер Celery	TCP 6379 / RESP	Передаёт задачи worker-процессам
MinIO	S3-совместимое объектное хранилище	HTTP 9000/9001	Хранит Parquet-файлы в bucket bronze
DuckDB	Встроенный аналитический движок	httpfs/S3	Читает API и пишет Parquet

2.3 Потоки данных и протоколы взаимодействия

Поток данных начинается с запуска DAG по расписанию. Airflow Scheduler создаёт экземпляр DAG run и помещает задачу в очередь Celery. Worker получает задачу через Redis, исполняет Python-код и запускает SQL-команды DuckDB. DuckDB обращается к OpenSky Network API по HTTPS на порт 443, получает JSON-ответ метода /api/states/all и разворачивает массив states с помощью функции unnest.

После чтения API DuckDB через расширение httpfs подключается к S3-совместимому endpoint minio:9000. Параметр s3_url_style устанавливается в path, что подходит для локального MinIO. Также указываются access key, secret key и отключение SSL, поскольку взаимодействие между

контейнерами происходит внутри локальной Docker-сети по HTTP. Результат сохраняется командой COPY TO в Parquet-файл внутри бакета bronze.

Отдельный поток технических данных связан с работой самого Airflow. Scheduler и webserver обращаются к PostgreSQL по протоколу PostgreSQL wire на порт 5432, чтобы записывать и читать метаданные. Worker и scheduler взаимодействуют с Redis по протоколу RESP на порт 6379. Пользовательские интерфейсы доступны по HTTP: Airflow UI на 8080 и MinIO Console на 9001. Таким образом, архитектура содержит как внешние протоколы обмена данными, так и внутренние протоколы управления выполнением задач.

2.4 Логическая структура DAG

DAG `raw_from_api_to_s3` описан в файле `dags/raw_from_api_to_S3.py`. Его владелец указан как `L.sudilovski`, идентификатор DAG — `raw_from_api_to_s3`, слой данных — `raw`, источник — `opensky`. В `default_args` настроены дата начала 1 мая 2026 года, `catchup`, три повторные попытки и задержка между повторами в один час. Расписание задано выражением `0 5 * * *`, то есть запуск выполняется ежедневно в 05:00.

Функция `get_dates` извлекает из контекста Airflow начало и конец интервала данных. Это важно, потому что Airflow работает не просто с текущей датой запуска, а с логическим интервалом обработки. В рассматриваемом DAG `start_date` используется в имени каталога и файла, благодаря чему результат каждого запуска получает отдельный путь в объектном хранилище. Такой подход облегчает проверку истории запусков и предотвращает смешивание разных снимков данных.

Центральная функция `get_and_transfer_api_data_to_s3` открывает соединение DuckDB, настраивает расширение `httpfs`, указывает параметры S3 и выполняет COPY-запрос. Внутри запроса данные читаются из `read_json_auto('https://opensky-network.org/api/states/all')`, поле `states` разворачивается в строки, после чего результат записывается в `s3://bronze/raw/opensky/<дата>/<файл>`. Конструкция `start >>`

`get_and_transfer_api_data_to_s3 >> end` делает зависимости явными и удобными для отображения в Airflow UI.

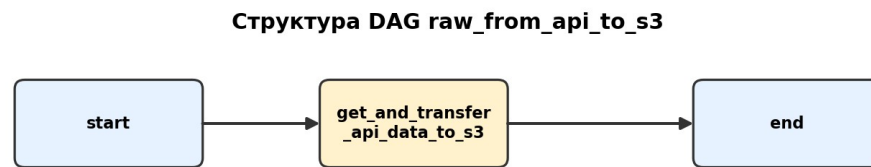


Рисунок 2 – Последовательность задач DAG raw_from_api_to_s3

2.5 Структура данных и правила именования объектов

OpenSky Network API возвращает объект JSON, содержащий время формирования ответа и массив `states`. Каждый элемент массива представляет собой `state vector` — упорядоченный список значений, относящихся к одному воздушному судну. В документации OpenSky описываются такие смысловые поля, как ICAO24-идентификатор, позывной, страна происхождения, временные метки, координаты, высота, признак нахождения на земле, скорость, курс, вертикальная скорость и другие параметры.

В текущей реализации массив `states` разворачивается в столбец `state` без детального именования каждого элемента. Для raw-слоя это допустимо, поскольку основная задача — сохранить исходный снимок без потери данных. Однако уже на уровне проектирования важно понимать, что следующий слой должен выполнить семантическую расшифровку массива. Это позволит заменить позиционный доступ к элементам массива на именованные колонки, упростить проверки качества и сделать данные пригодными для SQL-аналитики.

Правила именования объектов в S3 должны быть стабильными. В проекте используется шаблон `s3://bronze/raw/opensky/YYYY-MM-DD/YYYY-MM-DD_00-00-00.parquet`. Он содержит `bucket`, `слой`, `источник`, `партицию по дате` и `имя файла`. Такой шаблон полезен не только человеку, который

открывает MinIO Console, но и программным инструментам: они могут перечислять каталоги по дате и обрабатывать только нужные интервалы.

При промышленном развитии системы можно добавить дополнительные элементы пути: час запуска, регион данных, версию схемы или идентификатор загрузки. В учебном проекте это избыточно, потому что DAG выполняется один раз в день и сохраняет один снимок. Тем не менее выбранная структура совместима с будущим расширением и не требует изменения базового принципа хранения.

3 Реализация программного решения

3.1 Реализация получения и первичной обработки данных

Практическая реализация построена как минимальный, но завершённый DAG. В начале файла импортируются logging, duckdb, pendulum, классы DAG, Variable, EmptyOperator и PythonOperator. Использование Variable позволяет хранить ключи доступа к S3 в Airflow, а не в исходном коде. Это особенно важно, поскольку часть локальных файлов и секретов исключена из Git с помощью .gitignore: журналы, данные, .env, виртуальные окружения и файл cred.py не попадают в репозиторий.

Первичная обработка данных в данном проекте заключается не в бизнес-трансформации, а в техническом приведении ответа API к файловому аналитическому формату. OpenSky возвращает JSON-объект, где список состояний находится во вложенном поле states. DuckDB позволяет прочитать такой JSON напрямую и выполнить unnest(states), получив табличный результат. Сохранять исходный снимок в Parquet рационально: дальнейшие этапы смогут читать его быстрее и дешевле, чем исходный JSON.

Важным свойством решения является отсутствие промежуточных файлов на локальном диске. Данные читаются из HTTPS-источника и записываются непосредственно в MinIO через S3 API. Это уменьшает количество временных состояний и упрощает эксплуатацию. При этом

Airflow сохраняет логи выполнения, что позволяет отследить дату запуска, успешность операции и сообщения об ошибках.

Ключевой фрагмент SQL-команды DuckDB имеет следующий вид:

```
COPY (
    SELECT
        time,
        state
    FROM read_json_auto('https://opensky-network.org/api/states/all')
    TO 's3://bronze/raw/opensky/{start_date}/{start_date}_00-00-00.parquet';
```

3.2 Конфигурация доступа к объектному хранилищу

Доступ к MinIO в DAG задаётся через переменные Airflow `access_key` и `secret_key`. На уровне SQL DuckDB они передаются в параметры `s3_access_key_id` и `s3_secret_access_key`. В тексте курсовой работы реальные значения не приводятся: достаточно указать принцип хранения секретов и место их использования. Такой подход соответствует базовым требованиям безопасности и предотвращает попадание ключей в Git-репозиторий.

В локальной инфраструктуре endpoint S3 задан как `minio:9000`. Имя `minio` является DNS-именем сервиса внутри Docker Compose сети. Параметр `s3_use_ssl = FALSE` используется потому, что внутри локального стенда взаимодействие происходит по HTTP. Для промышленной эксплуатации это решение необходимо изменить: использовать TLS, отдельные сервисные учётные записи, ограниченные политики доступа к бакету и централизованное управление секретами.

Структура пути `s3://bronze/raw/opensky/<дата>/<дата>_00-00-00.parquet` отражает несколько проектных решений. Bucket `bronze` отделяет первичный слой от будущих слоёв обработки. Каталог `raw` указывает на минимальную степень преобразования. Каталог `opensky` фиксирует источник данных. Дата в пути позволяет хранить разные снимки и упрощает последующую партиционную обработку.

Таблица 2 – Функциональные и нефункциональные требования

Группа требований	Требование	Реализация
Функциональные	Получать данные из внешнего API	GET-запрос к OpenSky /api/states/all через DuckDB

Функциональные	Сохранять результат в объектное хранилище	COPY TO в s3://bronze/raw/opensky/...
Функциональные	Запускать процесс по расписанию	DAG Airflow с cron-расписанием 0 5 * * *
Надёжность	Повторять задачу при временных ошибках	retries = 3, retry_delay = 1 час
Безопасность	Не хранить ключи в Git	Airflow Variables и .gitignore
Наблюдаемость	Предоставлять логи и статусы	Airflow UI и логи задач
Расширяемость	Поддерживать дальнейшую обработку	Parquet в S3-совместимом raw/bronze-слое

3.3 Идемпотентность, расписание и повторные попытки

Идемпотентность в контексте ETL означает, что повторный запуск задачи за тот же интервал не должен приводить к неконтролируемому дублированию или противоречивому состоянию данных. В реализованном DAG результат записывается по детерминированному пути, зависящему от даты интервала. Поэтому повторный запуск за одну и ту же дату перезапишет или обновит тот же логический объект, а не создаст произвольное количество новых файлов с разными именами.

Расписание 0 5 * * * выбрано как ежедневный запуск в утреннее время. Для учебного прототипа этого достаточно, чтобы показать автоматизацию. Параметр catchup = True означает, что Airflow может создать запуски за пропущенные интервалы начиная с start_date. Это полезно при необходимости восстановить историю, но требует осторожности: при большом числе пропущенных дней система может создать много запусков подряд. Для контроля нагрузки дополнительно заданы concurrency = 1, max_active_tasks = 1 и max_active_runs = 1.

Повторные попытки настроены через retries = 3 и retry_delay = один час. Такая настройка оправдана для внешнего API: временные сетевые ошибки, ограничения или недоступность сервиса могут исчезнуть через некоторое время. Если ошибка связана с неправильным ключом доступа, отсутствующим бакетом или изменением схемы API, повторные попытки сами по себе проблему не решат, но дадут оператору понятный статус сбоя в Airflow UI.

3.4 Сравнение с исходным учебным проектом

Курсовой проект создан на основе `pet_project_earthquake`, но имеет более узкую область. В исходном проекте реализована полноценная демонстрационная аналитическая цепочка: загрузка данных землетрясений из USGS API в S3, перенос в PostgreSQL, построение витрин и визуализация в Metabase. В текущей работе сохранена идея lakehouse-подхода, Docker-инфраструктура и использование Airflow, но практическая реализация ограничена автоматической загрузкой данных OpenSky в объектное хранилище.

Такое ограничение соответствует теме курсовой работы. В ней требуется показать систему автоматизированного сбора и первичной обработки данных из внешнего API, а не строить полный DWH и BI-контур. Поэтому сервисы `postgres_dwh` и `metabase` в `docker-compose` закомментированы. Это уменьшает сложность запуска, снижает требования к ресурсам и позволяет подробнее рассмотреть именно первый этап конвейера.

Отличается и сам источник данных. Исходный проект использовал CSV-ответ USGS API с параметрами `starttime` и `endtime`. Курсовой проект использует JSON-ответ OpenSky `/api/states/all`, который представляет актуальный снимок воздушного пространства. В результате изменяется логика чтения: вместо `read_csv_auto` применяется `read_json_auto` и `unnest`. Однако общая идея остаётся прежней: Airflow планирует запуск, DuckDB выполняет чтение и запись, а S3-совместимое хранилище сохраняет результат для последующих этапов.

3.5 Порядок развёртывания и проверки результата

Развёртывание проекта начинается с подготовки локального окружения. В репозитории указан файл `req.txt` с зависимостями `apache-airflow==2.10.5` и `duckdb==1.2.2`, а основная инфраструктура описана в `docker-compose.yaml`. Для контейнерного запуска достаточно установить Docker и Docker Compose, после чего выполнить `docker-compose up -d` из

корня проекта. При первом запуске сервис airflow-init выполняет миграции базы метаданных и создаёт пользователя веб-интерфейса.

После запуска инфраструктуры необходимо открыть Airflow UI и убедиться, что DAG `raw_from_api_to_s3` обнаружен системой. Затем в Airflow Variables должны быть добавлены значения `access_key` и `secret_key`. Эти переменные используются задачей для подключения к MinIO. Если бакет `bronze` ещё не создан, его нужно создать через MinIO Console. В учебном стенде это выполняется вручную, но в дальнейшем создание бакета можно автоматизировать отдельным `init`-скриптом или инфраструктурным кодом.

Проверка результата состоит из нескольких шагов. Сначала в Airflow UI запускается DAG вручную либо ожидается запуск по расписанию. Затем проверяется, что задачи `start`, `get_and_transfer_api_data_to_s3` и `end` завершились успешно. После этого в MinIO Console открывается bucket `bronze` и проверяется наличие объекта в каталоге `raw/opensky` за соответствующую дату. Дополнительно можно прочитать Parquet-файл через DuckDB, чтобы убедиться, что файл не пустой и содержит ожидаемую структуру.

Важно, что результат проверки не должен требовать публикации секретов. В отчёте достаточно описать, что ключи настроены в Airflow Variables, а значения скрыты. Скриншоты или демонстрационные материалы при защите можно делать так, чтобы они показывали успешный статус DAG и наличие объекта в бакете, но не раскрывали значения `access key` и `secret key`.

3.6 Соответствие реализации поставленным требованиям

Разработанная система соответствует основной цели курсовой работы: она автоматизирует получение данных из внешнего API и сохраняет их в объектном хранилище. Автоматизация достигается за счёт расписания Airflow и DAG-модели. Первичная обработка выполняется DuckDB, который

преобразует JSON-ответ во внутреннее табличное представление и сохраняет результат в Parquet.

Требование воспроизводимости обеспечивается контейнерной инфраструктурой. Все основные сервисы описаны в одном `docker-compose.yaml`, а версии Airflow и DuckDB зафиксированы в `req.txt` и настройках образа. Это позволяет перенести проект на другой компьютер с установленным Docker и получить аналогичную структуру сервисов. Для учебной курсовой работы такая воспроизводимость является достаточной.

Требование безопасности выполнено на базовом уровне: секреты доступа к S3 не размещены в исходном коде и исключённые локальные файлы не добавляются в Git. Вместе с тем в разделе оценки указаны ограничения учебного стенда и меры, необходимые для промышленного использования: отдельные сервисные пользователи, минимальные права, TLS и secret backend.

Требование расширяемости также выполняется. Хотя проект реализует только загрузку в S3, структура `raw/opensky` и формат Parquet позволяют добавить последующие DAG без изменения текущего этапа. Поэтому курсовой проект можно рассматривать как первый модуль более широкой lakehouse-системы, в которой позже появятся ODS-слой, витрины и визуализация.

4 Оценка решения и направления развития

4.1 Безопасность и управление секретами

В проекте разделены исходный код и чувствительная конфигурация. Git-репозиторий содержит DAG, `docker-compose.yaml`, `requirements`-файл и README, но не содержит локальные данные, журналы, `.env` и файл `cred.py`. Такой подход отражён в `.gitignore` и позволяет не публиковать секреты или служебные артефакты. В Airflow ключи доступа к S3 хранятся как Variables и извлекаются во время выполнения DAG.

Для учебного стенда допустимы демонстрационные значения MinIO root user и password, указанные в docker-compose. Однако в реальной эксплуатации root-пользователь не должен использоваться приложениями. Следует создать отдельного пользователя или access key с минимальными правами, например только на запись в конкретный bucket и чтение нужных объектов. Также необходимо включить TLS, ограничить доступ к UI и консоли MinIO, настроить роли Airflow и хранить секреты во внешнем secret backend.

Отдельное внимание требуется к логам. В текущей реализации SQL-строка формируется с подстановкой ключей. Если такой запрос будет выведен в лог целиком, секрет может попасть в журнал. Поэтому в дальнейшем лучше использовать механизм DuckDB secrets или конфигурировать подключение таким образом, чтобы значения ключей не появлялись в текстах запросов. В рамках курсовой работы реальные ключи не приводятся и не сохраняются в документе.

4.2 Качество данных и первичные проверки

Так как курсовой проект реализует raw-слой, набор проверок качества данных минимален, но их можно формализовать. На этапе получения данных важно контролировать доступность API, наличие поля states в ответе, ненулевой объём результата и успешность записи объекта в MinIO. Эти проверки помогают отличить успешный запуск с данными от ситуации, когда API вернул пустой или некорректный ответ.

Для OpenSky часть значений может отсутствовать: не для каждого воздушного судна доступны координаты, скорость или высота. Это не является ошибкой raw-слоя, потому что система должна сохранять данные такими, какими они пришли из источника. Однако для последующих слоёв потребуются правила типизации, фильтрации и контроля полноты. Например, можно отдельно считать количество записей, долю строк без координат и время получения снимка.

Качество хранения проверяется через структуру бакета. После успешного запуска должен появиться объект по ожидаемому пути, соответствующему дате интервала. Дополнительно можно проверять размер файла, возможность прочитать Parquet обратно через DuckDB и соответствие базового набора колонок ожиданиям. В промышленной версии такие проверки можно оформить отдельными задачами Airflow или подключить специализированные инструменты data quality.

4.3 Эксплуатация, мониторинг и восстановление

Эксплуатация решения начинается с запуска `docker-compose up -d`. После инициализации пользователь открывает Airflow UI, проверяет наличие DAG `raw_from_api_to_s3` и при необходимости задаёт Variables `access_key` и `secret_key`. Затем DAG можно включить и запустить вручную либо дождаться запуска по расписанию. Результат проверяется в MinIO Console в bucket `bronze`.

Airflow предоставляет несколько уровней мониторинга. В графе DAG видно состояние задач, в списке запусков — история выполнения, в логах — технические сообщения Python-кода и ошибки. Для CeleryExecutor дополнительно может использоваться Flower, если включить соответствующий профиль Docker Compose. PostgreSQL и Redis являются внутренними компонентами, но их доступность влияет на работоспособность всей системы.

Восстановление после ошибок зависит от причины сбоя. Если недоступен внешний API, задачу можно перезапустить позже или дождаться `retry`. Если отсутствует bucket `bronze`, его нужно создать в MinIO. Если неверно заданы Variables, требуется обновить значения в Airflow. Если сбой произошёл после частичной записи объекта, повторный запуск за тот же интервал должен привести к формированию ожидаемого файла по тому же пути.

4.4 Ограничения и направления развития

Главное ограничение текущей реализации состоит в том, что она сохраняет снимок текущего состояния OpenSky, но не строит полный исторический контур обработки. Данные не загружаются в DWH, не типизируются на ODS-слое, не проходят расширенные проверки качества и не визуализируются. Эти ограничения осознанны, поскольку тема курсовой работы сфокусирована на автоматизированном сборе и первичной обработке.

Следующим этапом развития может стать добавление задачи проверки результата после записи в S3. Например, Airflow может прочитать созданный Parquet-файл и записать в лог количество строк. Затем можно добавить слой stg или ods, где массив state будет разобран на именованные поля: icao24, callsign, origin_country, time_position, last_contact, longitude, latitude, baro_altitude, on_ground, velocity, true_track и другие параметры. Это повысит пригодность данных для аналитики.

Для приближения к промышленной эксплуатации следует улучшить безопасность и надёжность: использовать отдельные сервисные ключи, secret backend, TLS, централизованный мониторинг, алерты, лимиты обращений к API и обработку ошибок схемы. Также возможно заменить локальный MinIO на облачное S3-совместимое хранилище, а Docker Compose — на Kubernetes или управляемую инфраструктуру.

4.5 Экономическая и организационная оценка решения

Выбранный технологический стек хорошо подходит для учебного проекта, потому что все основные компоненты доступны как открытое программное обеспечение и могут быть запущены локально. Apache Airflow, DuckDB, PostgreSQL, Redis и MinIO не требуют покупки лицензий для демонстрационного применения. Основные затраты связаны не с лицензированием, а с вычислительными ресурсами, временем настройки и сопровождением инфраструктуры.

Использование Docker Compose снижает организационную сложность: студенту или разработчику не нужно вручную устанавливать каждый сервис в операционную систему. Контейнеры можно остановить, пересоздать и перенести вместе с описанием инфраструктуры. Это особенно полезно для курсовой работы, где важна возможность показать результат на защите и объяснить назначение каждого компонента.

С точки зрения трудозатрат наиболее сложными являются не строки кода DAG, а правильная настройка взаимодействия сервисов. Необходимо понимать, как Airflow использует PostgreSQL и Redis, как DuckDB подключается к S3 endpoint, почему ключи должны храниться отдельно и как проверять результат в MinIO. Поэтому проект имеет образовательную ценность: он демонстрирует не только программирование на Python, но и инженерное мышление при построении data pipeline.

Если переносить решение в промышленную среду, экономическая оценка изменится. Появятся расходы на облачное хранилище, вычислительные ресурсы, мониторинг, резервное копирование и безопасность. Однако сама архитектурная идея останется применимой: оркестратор управляет процессом, вычислительный движок выполняет обработку, объектное хранилище содержит raw-данные, а метаданные и логи позволяют контролировать выполнение.

ЗАКЛЮЧЕНИЕ

В ходе курсовой работы была разработана и описана система автоматизированного сбора и первичной обработки данных из внешнего API с использованием Apache Airflow и объектного хранилища. Практическая реализация сфокусирована на загрузке данных OpenSky Network API в S3-совместимое хранилище MinIO. Система разворачивается локально с помощью Docker Compose и включает Apache Airflow, PostgreSQL, Redis, MinIO и DuckDB.

В теоретической части рассмотрены особенности автоматизированного сбора данных, роль внешних API, назначение Apache Airflow как оркестратора и преимущества объектного хранения в формате Parquet. В проектной части описана архитектура компонентов, протоколы взаимодействия, структура Docker-инфраструктуры и логика DAG `raw_from_api_to_s3`. В практической части показано, как DuckDB читает JSON-ответ API, разворачивает поле `states` и записывает результат в bucket `bronze` по дате интервала.

Поставленные задачи выполнены. Проанализирован исходный учебный проект `pet_project_earthquake` и выделена часть, соответствующая теме курсовой работы. Обоснован выбор технологий, описаны настройки расписания, повторных попыток, доступа к S3 и структуры хранения. Отдельно рассмотрены вопросы безопасности: реальные ключи не помещаются в Git и должны храниться в Airflow Variables или специализированном хранилище секретов.

Разработанное решение имеет учебный характер, но отражает принципы, применимые в реальных data engineering системах: разделение ролей компонентов, воспроизводимая контейнерная инфраструктура, raw-слой в объектном хранилище, наблюдаемость через Airflow и расширяемая структура путей. В дальнейшем проект может быть дополнен проверками качества, разбором state vector на именованные поля, загрузкой в DWH, построением витрин и визуализацией данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Apache Airflow Documentation. Dags [Электронный ресурс]. – Режим доступа: <https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/dags.html>. – Дата доступа: 20.05.2026.
2. Apache Airflow Documentation. Running Airflow in Docker [Электронный ресурс]. – Режим доступа:

- <https://airflow.apache.org/docs/apache-airflow/stable/howto/docker-compose/index.html>. – Дата доступа: 20.05.2026.
3. DuckDB Documentation. httpfs extension and S3 API [Электронный ресурс]. – Режим доступа: https://duckdb.org/docs/stable/core_extensions/httpfs/s3api.html. – Дата доступа: 20.05.2026.
 4. MinIO Documentation. Object Store [Электронный ресурс]. – Режим доступа: <https://docs.min.io/community/minio-object-store/>. – Дата доступа: 20.05.2026.
 5. OpenSky Network API Documentation [Электронный ресурс]. – Режим доступа: <https://opensky-network.org/data/api-docs>. – Дата доступа: 20.05.2026.
 6. Korsakov I. Pet project earthquake [Электронный ресурс]. – Режим доступа: https://github.com/k0rsakov/pet_project_earthquake. – Дата доступа: 20.05.2026.
 7. Apache Software Foundation. docker-compose.yaml for Airflow 2.10.5 [Электронный ресурс]. – Режим доступа: <https://airflow.apache.org/docs/apache-airflow/2.10.5/docker-compose.yaml>. – Дата доступа: 20.05.2026.
 8. ГОСТ 7.32–2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления.
 9. ГОСТ 7.1–2003. Библиографическая запись. Библиографическое описание. Общие требования и правила составления.
 10. Методические рекомендации по оформлению курсовых и дипломных работ / Институт бизнеса БГУ. – Минск : БГУ, 2020. – 36 с.
 11. Официальный репозиторий курсового проекта course_project [Электронный ресурс]. – Режим доступа: https://github.com/SailorVAC/course_project. – Дата доступа: 20.05.2026.

12. Apache Airflow. Concepts and architecture [Электронный ресурс]. – Режим доступа: <https://airflow.apache.org/docs/>. – Дата доступа: 20.05.2026.

ПРИЛОЖЕНИЕ А

Фрагмент DAG raw_from_api_to_s3

```
DAG_ID = "raw_from_api_to_s3"
LAYER = "raw"
SOURCE = "opensky"
ACCESS_KEY = Variable.get("access_key")
SECRET_KEY = Variable.get("secret_key")

args = {
    "owner": "L.sudilovskiy",
    "start_date": pendulum.datetime(2026, 5, 1, tz="Europe/Moscow"),
    "catchup": True,
    "retries": 3,
    "retry_delay": pendulum.duration(hours=1),
}

COPY (
    SELECT
        time,
        state
    FROM read_json_auto('https://opensky-network.org/api/states/all')
    TO 's3://bronze/raw/opensky/{start_date}/{start_date}_00-00-00.parquet';
```

ПРИЛОЖЕНИЕ Б

Чек-лист проверки работоспособности прототипа

№	Проверка	Ожидаемый результат
1	Контейнеры Airflow, PostgreSQL, Redis и MinIO запущены через docker-compose.	Условие выполнено при корректной настройке стенда.
2	В Airflow UI доступен DAG raw_from_api_to_s3.	Условие выполнено при корректной настройке стенда.
3	В Airflow Variables заведены access_key и secret_key без публикации значений в Git.	Условие выполнено при корректной настройке стенда.
4	В MinIO создан bucket bronze.	Условие выполнено при корректной настройке стенда.
5	Задача get_and_transfer_api_data_to_s3 завершается со статусом success.	Условие выполнено при корректной настройке стенда.
6	В bucket bronze появляется объект raw/opensky/<дата>/<файл>.parquet.	Условие выполнено при корректной настройке стенда.
7	Parquet-файл может быть прочитан аналитическим инструментом, например DuckDB.	Условие выполнено при корректной настройке стенда.